

Umgang mit dem Kursmaterial

- Sämtliches Kursmaterial (Texte, Folien, Abbildungen, Aufgaben, Lösungen usw.) unterliegt dem Urheberrecht des Dozenten und/oder weiterer Rechteinhaber.
- Die Nutzung ist ausschließlich den Studierenden der angegebenen Fachhochschule im Rahmen des angegebenen Kurses und nur zu Ausbildungszwecken gestattet.
- Jede weitergehende Nutzung, insbesondere das Veröffentlichen, Vervielfältigen, Verbreiten, Teilen oder öffentliche Zugänglichmachen des Materials (z.B. auf Webseiten, in Cloud-Speichern, per E-Mail, in Präsentationen oder Publikationen), ist ohne ausdrückliche schriftliche Zustimmung untersagt und kann rechtliche Konsequenzen nach sich ziehen.
- Die Nutzung des Kursmaterials in Verbindung mit Sprachmodellen oder KI-Systemen (z.B. LLMs wie ChatGPT) ist nur dann zulässig, wenn sichergestellt ist, dass die Inhalte nicht zum Training, zur Speicherung oder zu einer sonstigen Weiterverarbeitung durch Dritte verwendet werden (wie z.B. dem LLM-Anbieter) und alle oben genannten Nutzungsbeschränkungen eingehalten werden.



Kapitel_02

Linux 1: Einführung

Betriebssysteme

Prof. Dr. Claus Fühner

Fakultät Informatik

Ostfalia Hochschule für angewandte Wissenschaften

Braunschweig/Wolfenbüttel

In dieser Woche: Praktikum Linux-Kommandozeile

- Aufgaben zur Linux Kommandozeile
- erster von drei Terminen für insgesamt 10% der Prüfungsleistung
- Sie müssen Ihre Lösung im Gespräch erklären können!
- Entscheidend ist die Erklärung im Gespräch, nicht (nur) die korrekte Lösung!
- Abgaben nur in der entsprechenden Woche – keine verspäteten Abgaben möglich!
- Nur für Krankheitsfälle:
Ein Sammeltermin für Nachhol-Testate am Ende der Vorlesungszeit

Literaturhinweise zu Linux

Viele Bücher in der FH-Bibliothek und im Buchhandel erhältlich, wir interessieren uns für die Kommandozeile

- http://de.wikipedia.org/wiki/Geschichte_von_Linux - Infos zur Entwicklung von Linux, der endlose Rechtsstreit mit SCO, usw.



man – auf Ihrem Linux-System!

Plötner/Wendzel: Linux – das umfassende Handbuch, Galileo/Rheinwerk
(auch unter <http://openbook.rheinwerk-verlag.de/linux/>)

- <http://www.bin-bash.de/>
- SelfLinux Tutorial: <http://www.selflinux.org/>



Gamification: <https://overthewire.org/wargames/bandit/>



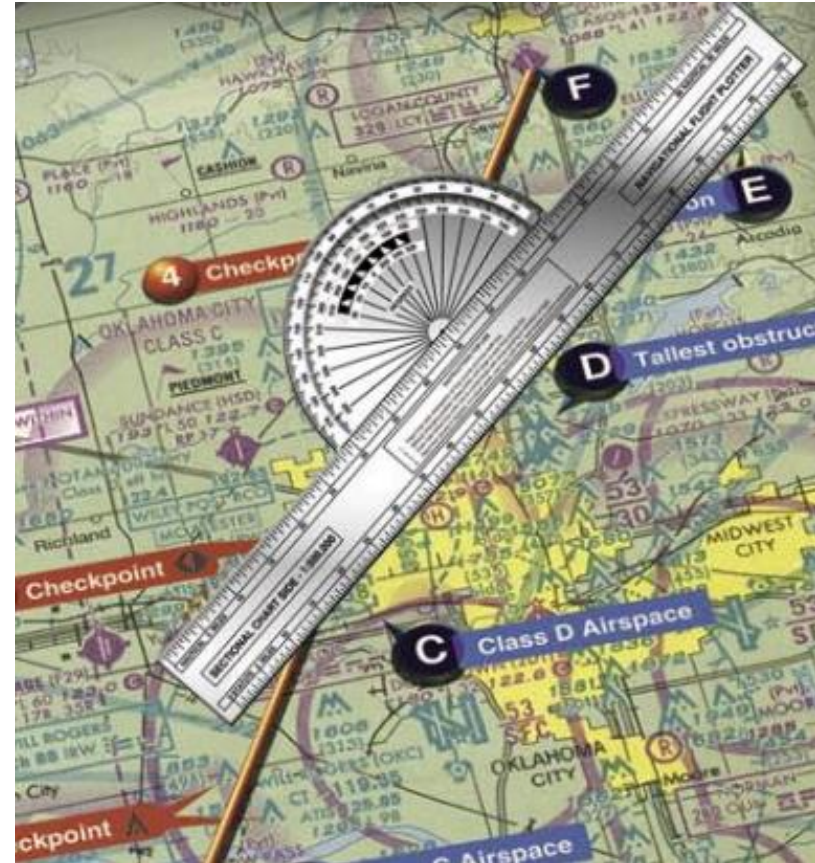
Agenda



Linux 1: Einführung

➔ Was ist Linux?

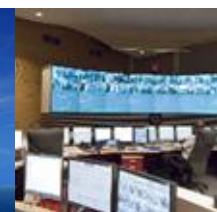
- Unix-Dateisystem
- Erste Schritte mit der Shell
- Selbsthilfe



Nur zu Lehrzwecken



Betriebssysteme



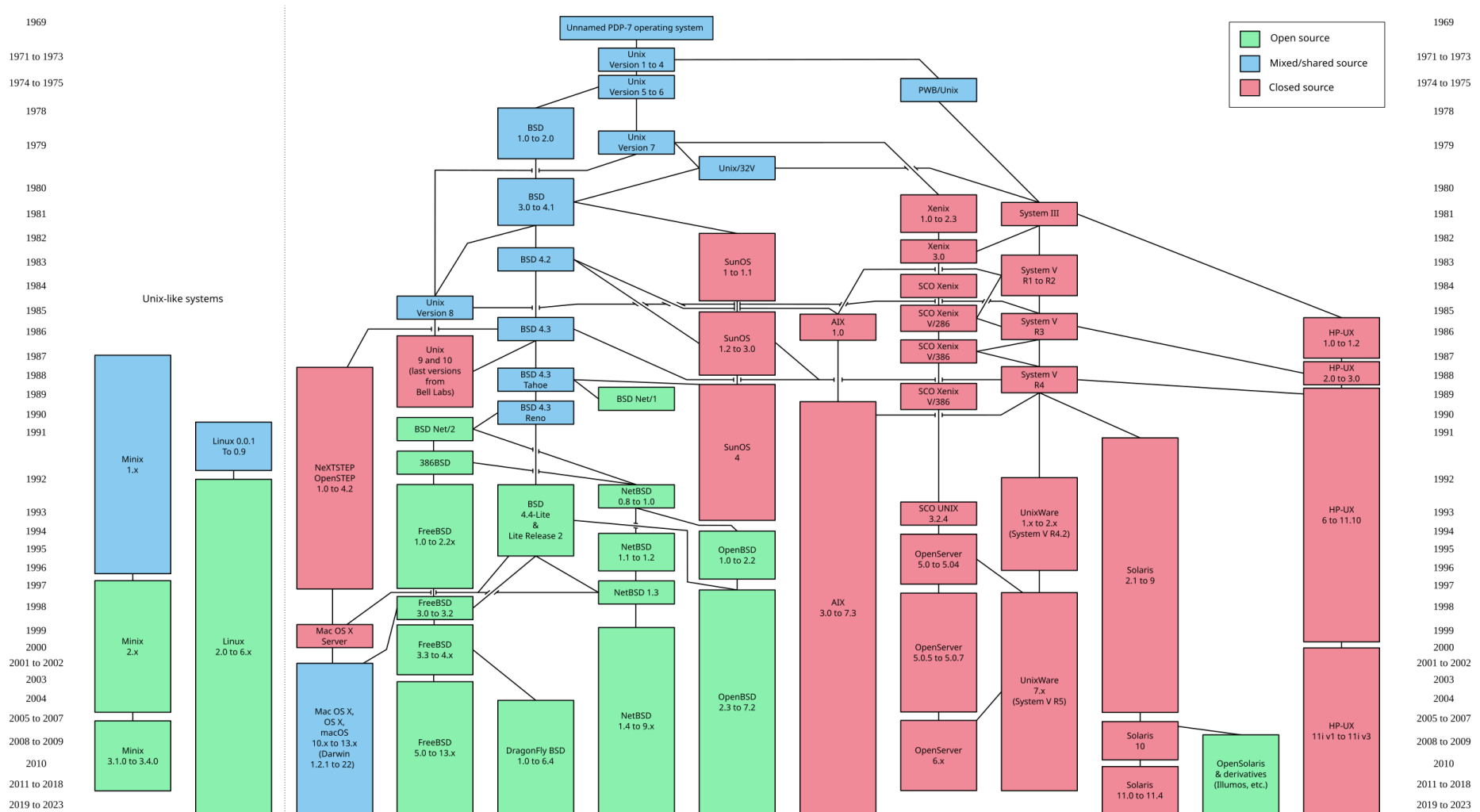
Wie alles begann: UNIX

- **UNIX** ist ein Betriebssystem, dessen Konzepte sich als **wegweisend** für viele Betriebssysteme herausstellen sollten.
- 1970 bei Bell Labs auf PDP-7 von Ken Thompson und Dennis Ritchie zunächst als privates Projekt (!) entwickelt
- Bis **UNIX** V7 1979 wurde UNIX mit einem C-Compiler kostenfrei an Hochschulen abgegeben → **frei** und portabel
- Aus Bell ging AT&T hervor und in den frühen 1980ern wurde UNIX **kommerzialisiert** und als UNIX System V vermarktet
- Recht komplizierte weitere Geschichte → <http://de.wikipedia.org/wiki/Unix>
- Heute gibt es **viele Unixe**: (Open)Solaris, AIX (IBM), HP-UX, Minix, Mac OS X, iOS, Android, Linux, ...



Was ist Linux?

Der UNIX-Stammbaum



Quelle: http://commons.wikimedia.org/wiki/File:Unix_history-simple.png by Eraserhead1, Infinity0, Sav_vas

GNU

- 1983 wurde das GNU-Projekt von **Richard Stallman** initiiert, GNU steht für „**GNU's Not Unix**“
- Projektziel war die Schaffung eines *freien* UNIX-kompatiblen Betriebssystems.
- **Freie Software** wird von der Free Software Foundation definiert als [\[http://www.gnu.org/philosophy/free-sw.html\]](http://www.gnu.org/philosophy/free-sw.html):
 - Freiheit 0:** Das Programm zu jedem Zweck auszuführen.
 - Freiheit 1:** Das Programm zu studieren und zu verändern.
 - Freiheit 2:** Das Programm zu verbreiten.
 - Freiheit 3:** Das Programm zu verbessern und zu verbreiten, um damit einen Nutzen für die Gemeinschaft zu erzeugen.
- **GNU General Public License (GPL):** das Programm muss *ohne Einschränkung der Freiheiten* weitergegeben werden („**Copyleft**“).
- Bis 1990 waren der GNU-C-Compiler (gcc), eine Shell (bash) und viele Kommandos entwickelt, es fehlte aber noch der Kernel.



Quellcode
erforderlich!

Minix und Linux

- 1987 veröffentlichte **Andrew S. Tanenbaum** den Unix-Klon **Minix**, zu dem Mikrokern, C-Compiler, Texteditor und viele Kommandos gehörten und das auf **einfacher PC-Hardware** lief (dies war bei UNIX System V nicht der Fall).
- Der Quellcode gehörte zu einem Buch → kommerziell und proprietär, aber sehr niedriger Preis.
- 1991 stellte **Linus Torvalds** einen Kernel vor, der von Minix inspiriert war, auf einem 386er PC lief und **Linux** hieß.
- Anfang 1992 wurde Linux unter die GNU General Public License GPL gestellt.
Torvalds sagte später:
„Making Linux GPL'd was definitely the best thing I ever did.“
→ der Erfolg von Linux begann



GNU/Linux

- Als **GNU/Linux** (oder kurz: **Linux**) bezeichnet man den
 - Linux-Kernel und die
 - GNU-Software (Compiler, Shell, Kommandos), durch die erst ein vollständiges Betriebssystem entsteht
- GNU/Linux ist
 - unixoid (unixartig), d.h. teilt wesentliche Merkmale von UNIX, stammt aber *nicht* vom Quellcode von UNIX ab.
 - freie Software
 - primär kommandoorientiert, aber auch mit unterschiedlichen Arbeitsplatzoberflächen („Desktops“) bedienbar
 - außerordentlich gut *skalierbar* (ab ca. 2 MB)
 - für *sehr* viele Plattformen verfügbar (Auswahl):
IA-32, **IA-64**, Sun SPARC, PowerPC, **ARM**, IBM S/390, MIPS, **RISC-V**

Was ist Linux?

Wie sieht dieses Unix aus?



Terminal

Shell



Was ist Linux?

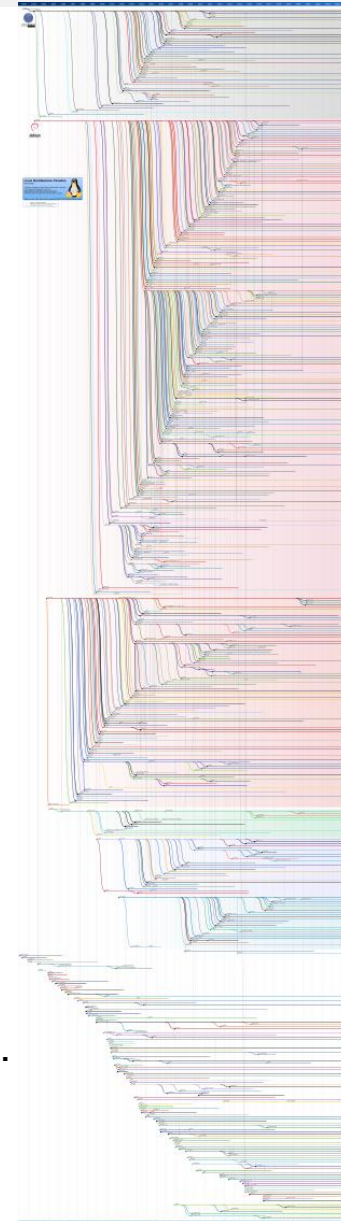
Desktops

- Nutzung über das Terminal oder graphische Desktops wie **GNOME**, **KDE Plasma**, **Xfce**, ...
- **Desktop Environments** bieten
 - Gestaltungsrichtlinien, Icons und Programmierschnittstellen (APIs) mit entsprechenden Libraries für ein einheitliches **Look & Feel** aller Anwendungen
 - Verschiedene Anwendungen wie z.B. File Manager, Multimedia Player, Photo Manager und Tools für Systemeinstellungen.
 - Z.B. KDE-Anwendungen lassen sich auch beispielsweise in Gnome nutzen, behalten dann aber das KDE-spezifischem Look-And-Feel
- Gemeinsame Grundlage für die verschiedenen Desktops ist das **X Window System** bzw. (neuer) **Wayland**. Dies verwaltet Tastatur, Maus und Bildschirm und stellt die Primitiven für graphische Benutzungsoberflächen (GUIs) bereit.
- große Ausnahme: MacOS



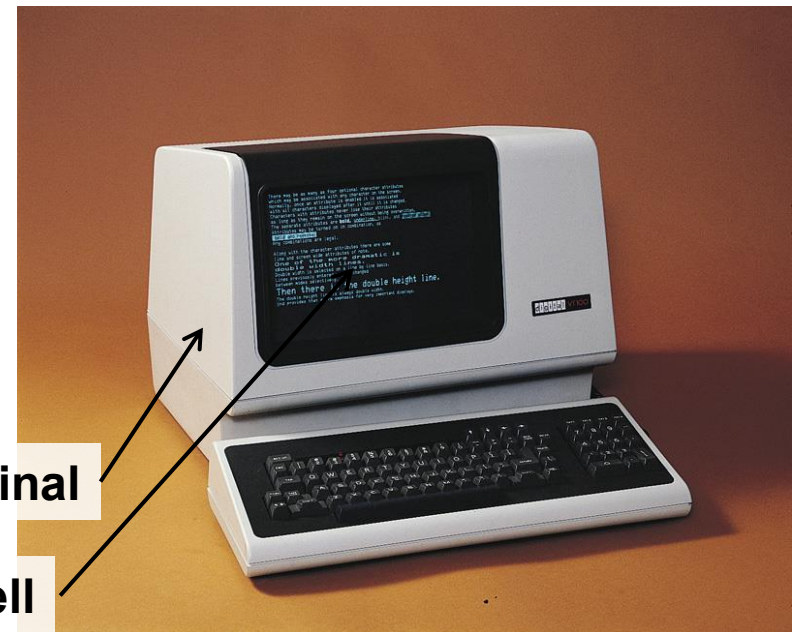
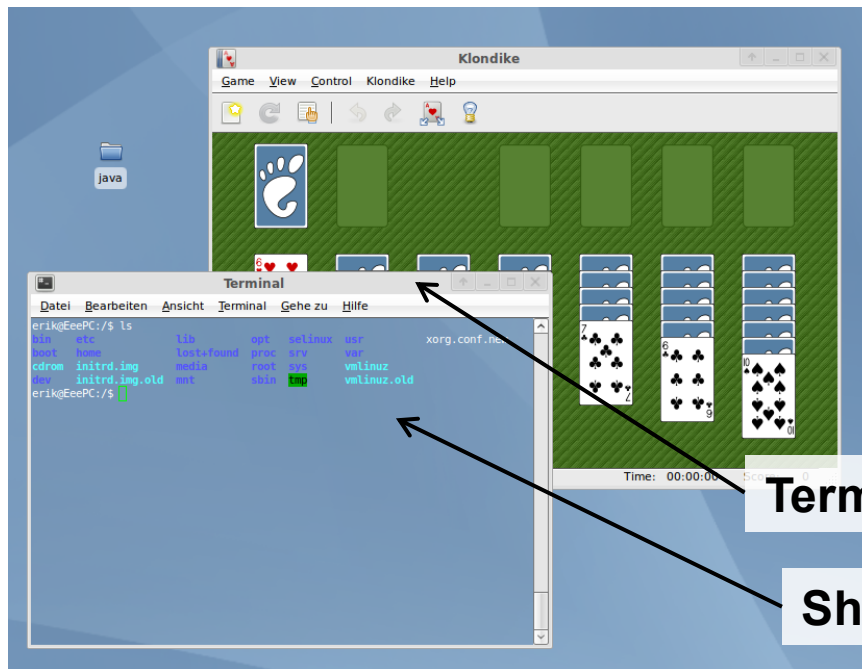
Linux-Distributionen

- **Linux-Distributionen** fassen die Bausteine zusammen, so dass ein komplettes Linux *einfach* installiert werden kann
- Zu einer Distribution gehört
 - Linux Kernel
 - GNU Software für elementare Kommandos
 - Desktop Environment mit X Window System oder Wayland
 - Anwendungssoftware: Auswahl ähnlich Windows oder MacOS, aber leider kein Microsoft Office ☹ (aber LibreOffice und Online-Alternativen)
 - **Paketverwaltung** (zusätzliche Programme aus einem zentralen Distributions-Repository laden, Updates beziehen)
 - Boot-Manager (da meistens MS Windows vorinstalliert ist)
 - Treiber, Updates, Kernel-Patches, Support, ...
- Bekannte Distributionen
 - **Debian**, Ubuntu, Mint, Redhat, openSUSE, ArchLinux, Rocky Linux, ...
 - Smartphones: **Android**



Interaktion mit Unix – Terminal & Shell

- Mit dem Betriebssystem wird über
 - verschiedene **Desktopanwendungen** per *Maus* oder
 - einen **Kommandozeileninterpreter (Shell)** per *Tastatur* interagiert
- Die Shell wird in einem **Terminal** ausgeführt, d.h. in einer *Emulation* eines Terminal-Gerätes

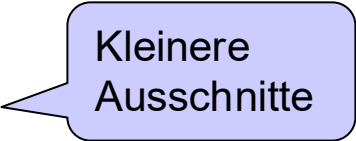


Unterschiedliche Sichten auf Unix

- Anwendersicht (und Grundlage der anderen Sichten)
 - Dateien und Verzeichnisse – das Dateisystem
 - Externe Geräte
 - Prozessverwaltung
 - Kommandozeilen-Interpreter („Shell“)
 - Selbsthilfe
- Administratorsicht
 - Installation, Boot-Vorgang und Systemstart
 - Benutzerverwaltung
 - System- und Netzwerkkonfiguration
 - Sicherheit, Optimierung
- Entwicklersicht
 - Unix-Anwendungsentwickler: Systemaufrufe
 - Kernel-Entwickler: ...



Unser
Fokus



Kleinere
Ausschnitte



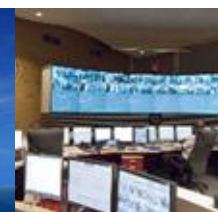
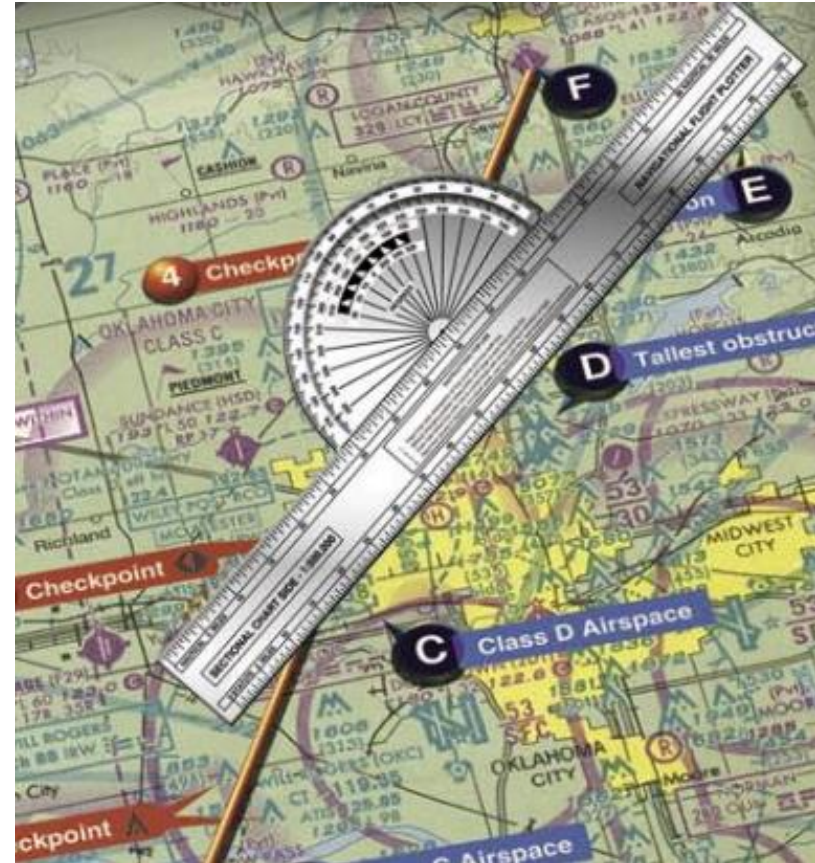
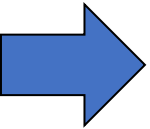
Kleinere
Ausschnitte

Agenda



Linux 1: Einführung

- Was ist Linux?
- Unix-Dateisystem
- Erste Schritte mit der Shell
- Selbsthilfe



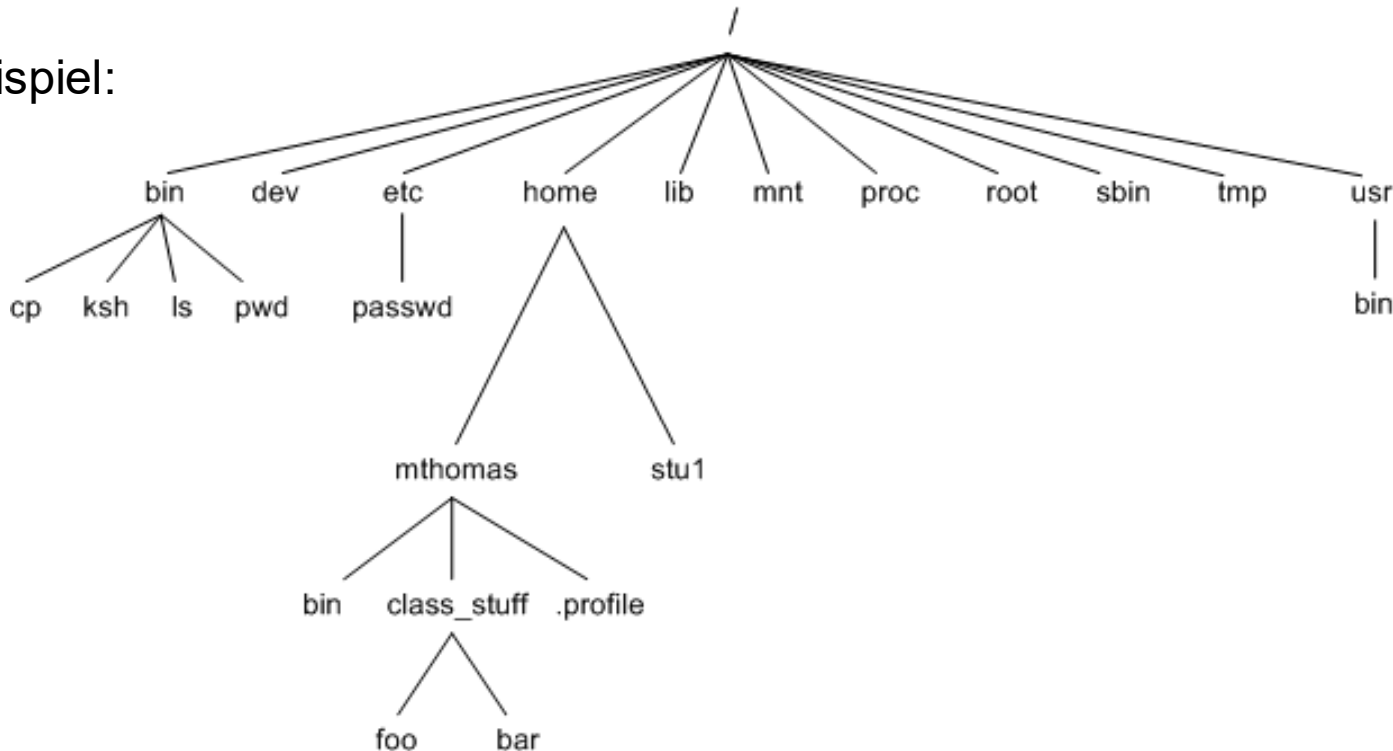
Der Unix-Dateibaum

- Dateien sind die *elementare Organisationseinheit* für Daten.
- Dateien dienen dazu, Informationen *persistent* abzuspeichern.
- Dateien werden *hierarchisch* in *einem Verzeichnisbaum* organisiert
- Alle *Datenträger* (z.B. Festplatten, SSDs, aber auch *temporäre Medien* wie USB-Sticks) und *Netzwerkressourcen* (z.B. Verzeichnisse auf entfernten Rechnern) werden in diesen Verzeichnisbaum eingebunden:
 - *integrierte Sicht* über alle Datenträger und angeschlossenen Geräte
 - es gibt *keine* Laufwerksbuchstaben (Unterschied zu Windows)
- Geräte wie z.B. SSDs oder USB Sticks an eine beliebige Stelle im Verzeichnisbaum einhängen (erfordert meist Rechte als *Superuser*):
 - einhängen `mount /dev/sdc1 /mnt/my_mount_point`
 - aushängen `umount /mnt/my_mount_point`
- Typ des Dateisystems mit `-t`, davon abhängig Optionen wie *readonly* mit `-o`
- Defaults in `/etc/fstab`



Der Unix-Dateibaum

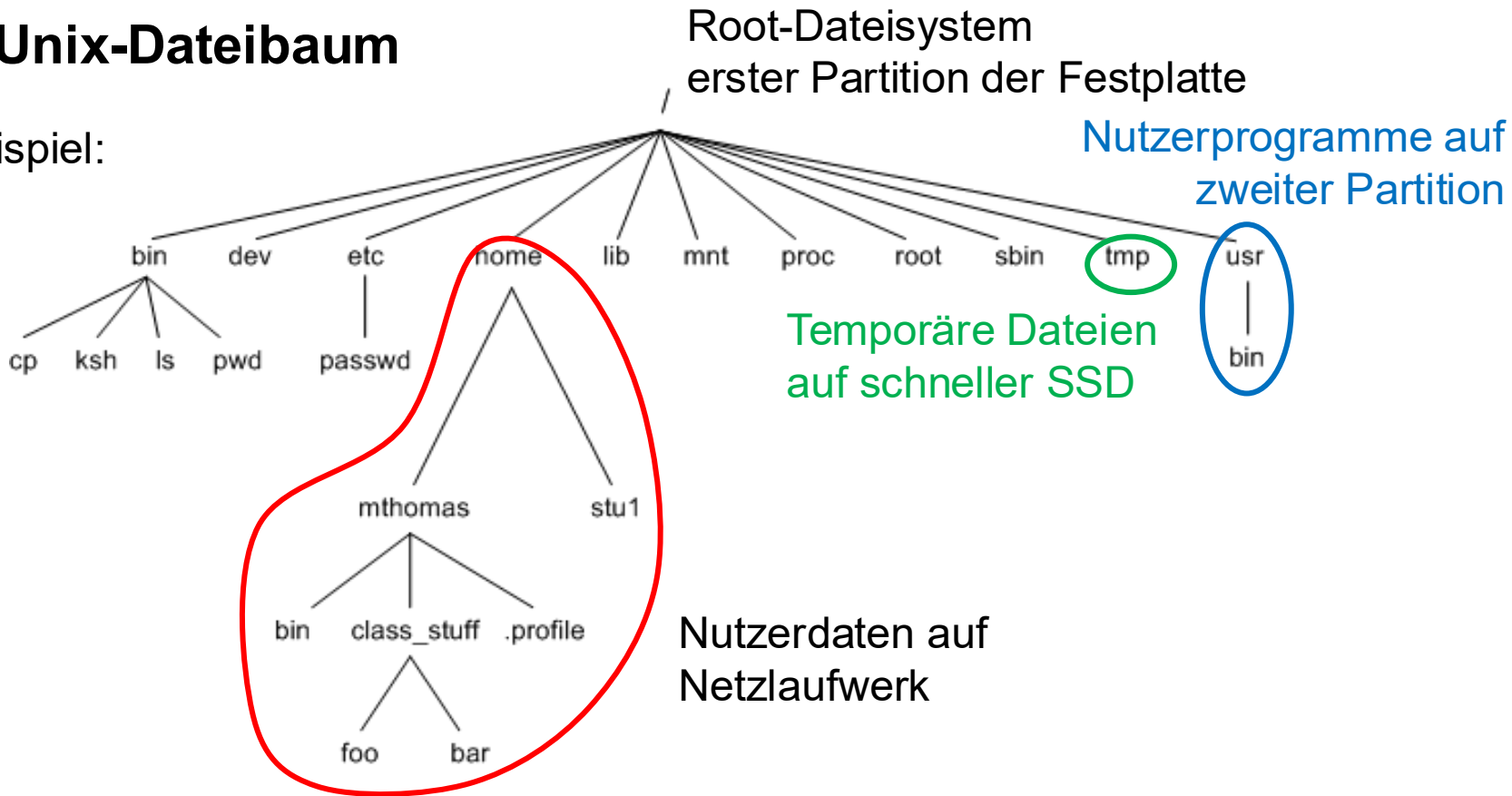
- Beispiel:



- Der Aufbau des Dateibaums wird durch die Unix File System Hierarchy (FHS) standardisiert
- Aktueller Stand siehe: <http://www.pathname.com/fhs/>

Der Unix-Dateibaum

- Beispiel:



- Keine Laufwerksbuchstaben
- Geräte werden an beliebiger Stelle im Dateibaum eingehängt

Der Unix-Dateibaum

Verzeichnis	Beschreibung
/	Wurzelverzeichnis
/bin	binaries - Unix-Kommandos für normale Benutzer
/dev	devices - Gerätedateien (Terminal, Festplatten, USB Stick, Kamera, ...)
/etc	etcetera - systemweite Konfigurationsdateien
/home	Home-Verzeichnisse der Benutzer
/lib	libraries - gemeinsam genutzte Bibliotheken
/opt	optional - optionale Anwendungsprogramme
/proc /sys	Abbildung von Systeminformationen auf das Dateisystem
/root	Home-Verzeichnis des Superusers root
/sbin	system binaries - Kommandos zur Systemverwaltung
/tmp	temporaries - temporäre Dateien
/usr	unix system resources - Anwendungsprogramme, X Window System
/var	variable - veränderliche Dateien

Pfadangaben

- *Slash /* dient als **Trennzeichen** (nicht in Dateinamen erlaubt)
- **Absolute Pfadangabe** startet mit Slash /
z.B. /etc/passwd
- **Relative Pfadangabe** zum aktuellen Verzeichnis,
z.B. ../etc/passwd, test.md oder ./demo/test3.c
- Absolute Pfadangabe ab dem **Home-Verzeichnis**,
z.B. ~/.../etc/passwd

Pfade: Ausgezeichnete Verzeichnisse

Bezeichnung	Englisch	Kürzel
Wurzelverzeichnis	Root Directory	/
Aktuelles Verzeichnis	Working Directory	.
Heimatverzeichnis eines Benutzers	Home Directory	~
Übergeordnetes Verzeichnis	Parent Directory	..
Vorhergehendes Verzeichnis		-

Benennung von Dateien/Verzeichnissen

Allgemeine Unix-Eigenschaften (alle Dateisysteme)

- Dateiextension ganz normaler Teil des Dateinamens ohne Einschränkungen, z.B.: `file.tar.gz`
- Unix **wertet Extensionen nicht aus**
 - Verknüpfung von Dateityp und bevorzugter Anwendung ist komplizierter (Heuristik basierend auf Extension, Dateiattributen und Dateiinhalt)
- Dateien und Verzeichnisse können **versteckt** werden, indem ihnen ein `.` („dot“) voran gestellt wird, z.B. `.bashrc`
- Der Befehl `ls` zeigt solche Dateien ohne weitere Optionen nicht an!
- Für verschiedene Anwendungsbereiche gibt es verschiedene Konventionen, z.B.: Softwareentwicklung:
 - keine Leerzeichen
 - keine Sonderzeichen
 - Konvention zur Verwendung Extensions, z.B. `xy.java` für Java-Quellcode

Benennung von Dateien/Verzeichnissen (ext4)

Eigenschaften des Dateisystems ext4 (typisches Dateisystem)

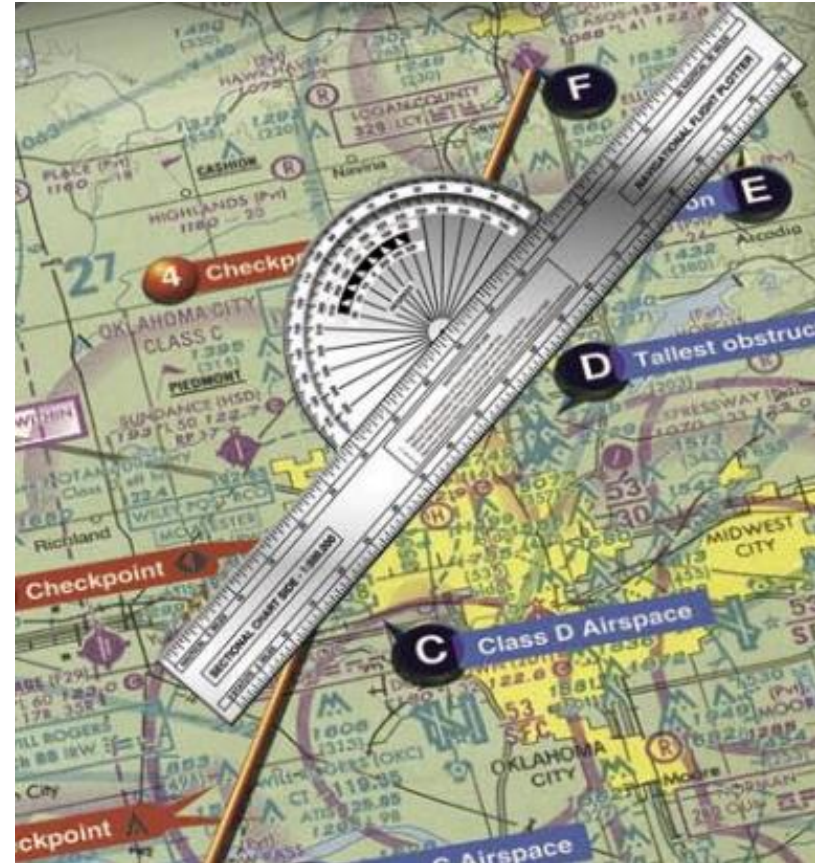
- Maximale Länge von Datei- und Verzeichnisnamen beträgt **255 Zeichen** (Unicode UTF-8)
- Es wird zwischen **Groß- und Kleinschreibung** **unterschieden** (dies ist bei Windows nicht der Fall!)
- Alle Zeichen **außer NUL** (**0x00**) und Slash **/** erlaubt
 - Nutzung aus Programmen unproblematisch
 - **Aber:** Einige Sonderzeichen werden von der Shell interpretiert
→ Müssen in Shell-Kommandos geschützt werden (**Quoting, später**)

Agenda



Linux 1: Einführung

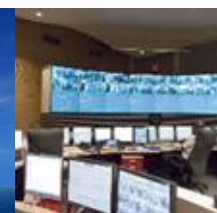
- Was ist Linux?
- Unix-Dateisystem
- Erste Schritte mit der Shell
- Selbsthilfe



Nur zu Lehrzwecken



Betriebssysteme



Die Shell – der Unix-Befehlsinterpreter

- **Verschiedene Shells** möglich (csh, ksh, **bash**, zsh)
- Interaktive Eingabe von Kommandos am Prompt
- Eingabe: **Befehl Option1 Option2 ... Argument1 Argument2 ...**
 - **Optionen** starten mit - (Kurzform) oder -- (Langform)
 - **Argumente** starten nicht mit - oder -- und sind meist Dateinamen, optional mit Wildcards
- Shell unterstützt verschiedene **Befehlstypen**:
 1. **Interne Befehle** (z.B. **cd**, **pwd**) einschl. Kontrollstrukturen
 2. Starten von **Skripten** (Textdateien mit Shell-Befehlen)
 3. **Externe Befehle** (d.h. Programmen) (z.B. **who**, **ls**)
- Befehlstyp anzeigen mit **type**

```
fuehner@CF-MBP23 ~ % type ls
ls is /bin/ls
fuehner@CF-MBP23 ~ % ls
academiccloud      Downloads          Music
anaconda.sh        git               nocloud
Applications       go                Pictures
```

Der Befehl ls

- `ls` – Inhalt eines Verzeichnisses ausgeben (*list*)
- Der `ls` Befehl kennt *Optionen*, z.B.
 - `-l` (“long”) - eine detaillierte Ausgabe
 - `-a` („all“) – auch versteckte Dateien ausgeben
 - `-R` („recursive“) – rekursive Ausgabe aller Unterverzeichnisse
- Nutzt sowohl *kurze* als auch *lange* Optionen
 - `-a` (kurz) und `--all` (lang) haben den gleichen Effekt
 - `--help` – Ausgabe von Hilfsinformation zum Kommando
 - `--color` – farbige Ausgabe des Verzeichnisinhalts
- Kurze Optionen können auch zusammengefasst werden
Beispiel: `ls -laR` (entspricht `ls -l -a -R`)
- `ls` listet den Inhalt des *aktuellen Verzeichnisses* oder der in der Argumentliste angegebenen Verzeichnisse auf

ls - Beispiel

- Vollständige Syntax: `ls [-optionen] [pfad]`

- Beispiel:

```
$ ls -l /
```

```
total 4194412
```

```
lrwxrwxrwx   1 root    root          7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x   4 root    root        4096 Mar  4 06:57 boot
dr-xr-xr-x   2 root    root        4096 Aug  5  2025 cdrom
drwxr-xr-x  21 root    root       4320 Feb 25 18:02 dev
drwxr-xr-x 127 root    root      12288 Feb 25 17:45 etc
drwxr-xr-x   6 root    root       4096 Aug 25  2025 home
lrwxrwxrwx   1 root    root          7 Apr 22  2024 lib -> usr/lib
lrwxrwxrwx   1 root    root          9 Apr 22  2024 lib64 -> usr/lib64
drwx-----  2 root    root      16384 Aug 12  2025 lost+found
drwxr-xr-x   2 root    root       4096 Aug  5  2025 media
drwxr-xr-x   2 root    root       4096 Aug 24  2025 mnt
drwxr-xr-x   4 root    root       4096 Aug 29  2025 opt
dr-xr-xr-x 344 root    root          0 Feb 25 18:02 proc
```

Interne (eingebaute) Shell-Befehle

Eingebaute Befehle sind in der Shell selbst implementiert, also nicht extern.

Auswahl für die bash:

- `pwd` Aktuelles Verzeichnis anzeigen (*print working directory*)
- `cd` Verzeichnis wechseln (*change directory*)
 - `cd` (ohne Argumente) wechselt in das Heimat-Verzeichnis
 - `cd ~user` wechselt in das Heimat-Verzeichnis den Nutzers user
- `alias` Synonyme für wiederkehrende Befehle definieren
- `echo` Ausgabe von Argumenten (auch Shell-Variablen)
- `type` Befehlstyp ausgeben bzw. Interpretation durch Shell bei alias
- `help` Hilfe zu den in die Shell *eingebauten* Befehlen
(über man nicht verfügbar!)
- `;` Gruppierung von Kommandos (werden nacheinander ausgeführt)

Befehle zu Dateien und Verzeichnissen

- Wir kennen schon: `pwd`, `cd`, `ls`

`cp` kopiert Dateien oder Verzeichnisse (*copy*)

`mv` verschiebt Dateien oder Verzeichnisse (*move*)

`rm` löscht Dateien oder Verzeichnisse (*remove*)

`touch` eine (leere) Datei erzeugen falls sie noch nicht existiert
bzw. deren Zeitstempel aktualisieren

`nano`, `vi`, `emacs`

Datei-Editoren mit Text-UI, laufen im Terminal
auch Editoren mit graphischem UI verfügbar, z.B. `code`

`less` Ausgabe längerer Dateien auf dem Bildschirm

`mkdir` ein Verzeichnis erzeugen

`rmdir` ein Verzeichnis löschen

Beispiele

```
cd /usr/bin  
ls  
cd ~  
mkdir demo  
cd demo  
nano hellounix  
ls -l  
less hellounix  
cd ..  
touch test  
mv ./demo ./demo2
```

Weitere elementare Befehle

- `which` zeigt zu einem Kommando die Programmdatei, die ausgeführt werden würde
- `file` Typ einer Datei (aus Heuristik über Dateiinhalt, nicht über Dateinamenserweiterung – die ist in Unix irrelevant)
- `ps` liefert die laufenden Prozesse mit ihren Prozess-IDs
- `kill` Abbruch eines Prozesses (auch mit CTRL-C möglich)
- `who, w` welche Benutzer sind angemeldet?
- `whoami` wer bin ich?
- `date` Systemzeit

Command History und Auto-completion

- **Command History:** die bash und andere Shells haben ein Befehlsgedächtnis
 - Mit den Cursortasten wird durch die Befehlshistorie gescrollt
 - Alte Befehle können editiert und nochmal ausgeführt werden
 - **history** zeigt alle Befehle mit ihrer Nummer
 - **!num** führt den Befehl mit der Nummer **num** nochmal aus
 - **!text** führt den letzten Befehl aus, der mit **text** beginnt
- **Auto-completion:** Namen von Befehlen, Dateien und Verzeichnissen müssen nicht komplett eingegeben werden, es reichen jeweils die ersten Buchstaben und ein **TAB**, die Shell zeigt dann mögliche Vervollständigungen
- Bash-Nutzer können in der Datei **~/.bashrc** Einstellungen vornehmen
 - Diese Kommandos werden beim Start einer neuen Shell automatisch als Skript ausgeführt
 - Dateiname ist Shell-spezifisch, z.B. zsh -> **~/.zshrc**

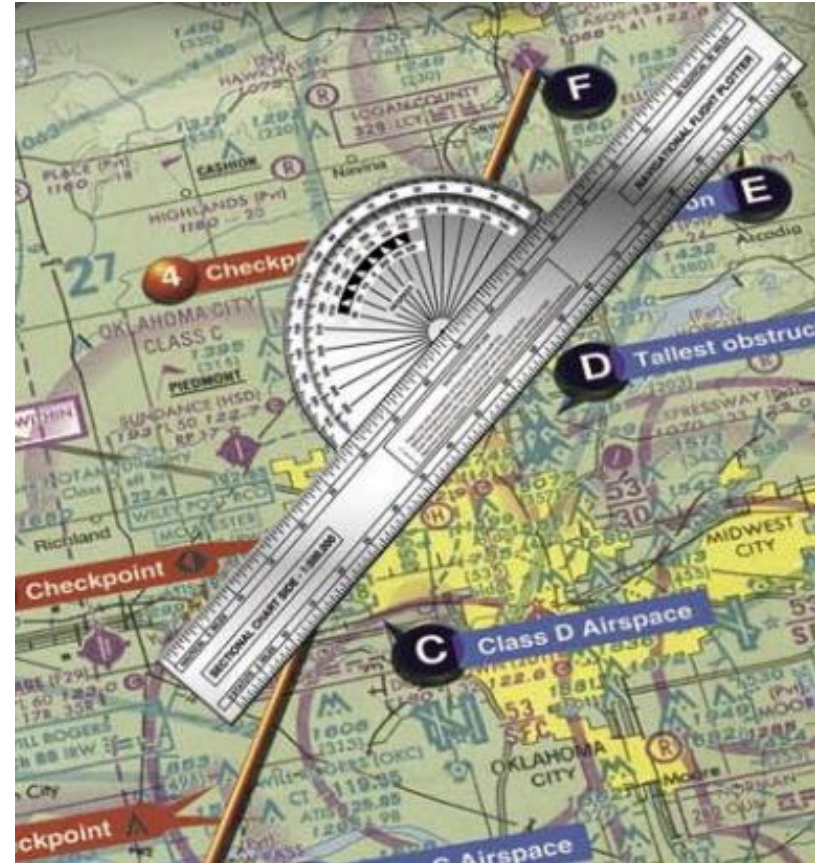
Agenda



Linux 1: Einführung

- Was ist Linux?
- Unix-Dateisystem
- Erste Schritte mit der Shell

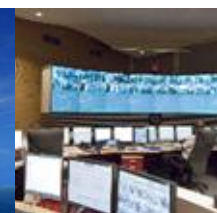
➔ Selbsthilfe



Nur zu Lehrzwecken



Betriebssysteme



Das Handbuch (Manual)



- Unix-Systeme kommen mit einem eingebauten Handbuch
- Manual-Pages aufrufen `man [Section] Befehlsname` (ausführlicher als -h)
- In Abschnitte (sections) gegliedert:

Section	Description
1	General commands
2	System calls
3	Library functions, covering in particular the C standard library
4	Special files (usually devices, those found in /dev) and drivers
5	File formats and conventions
6	Games and screensavers
7	Miscellanea
8	System administration commands and daemons



Das Handbuch (Manual)



```
MKDIR(1)                                User Commands                                MKDIR(1)

NAME
    mkdir - make directories

SYNOPSIS
    mkdir [OPTION]... DIRECTORY...

DESCRIPTION
    Create the DIRECTORY(ies), if they do not already exist.

    Mandatory arguments to long options are mandatory for short options
    too.

    -m, --mode=MODE
        set file mode (as in chmod), not a=rwx - umask

    -p, --parents
        no error if existing, make parent directories as needed

    -v, --verbose
        print a message for each created directory
```

- **Synopsis:** Kommando mit [optionalen] und Pflicht-Argumenten
- Scrollen mit den Pfeiltasten CursorUp/-Down und BildUp/-Down
- **q** zum Verlassen der Manual-Seite
- **/** zum Suchen auf der Seite (Suchbegriff + Return)

Weitere Informationsquellen



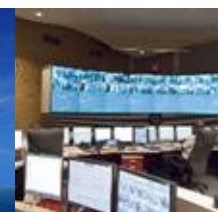
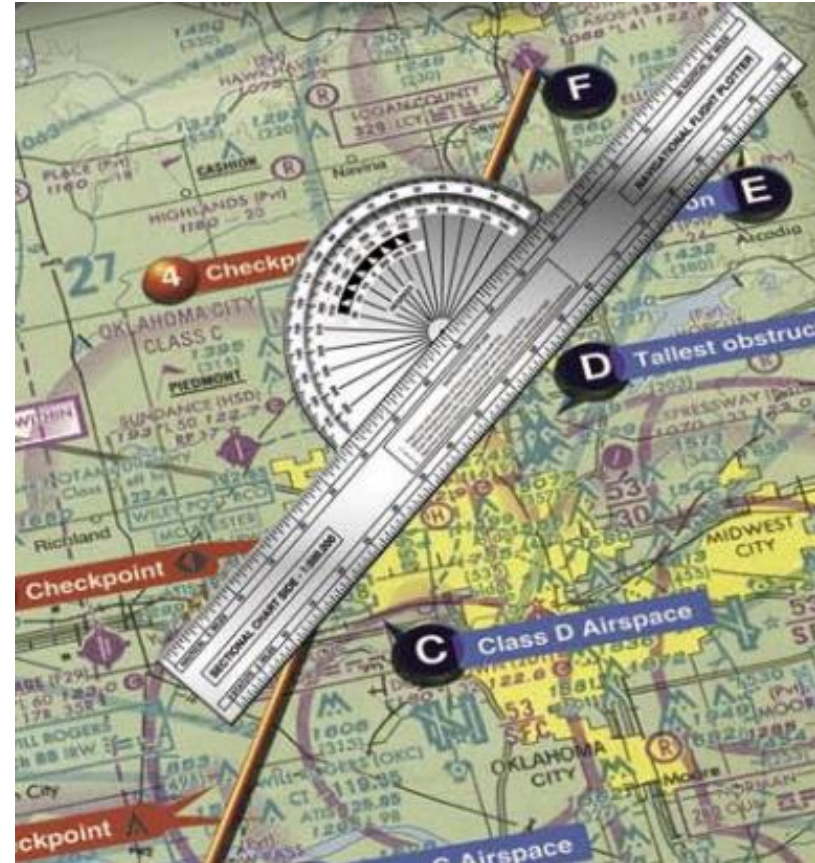
- **Shell-Interne Befehle** sind auf der Manalseite der Shell zu finden (oder auch über den bash-internen Befehl **help InternalCommand**)
- **man** gibt es auch im Web, z.B.: <http://linux.die.net/man/>
Vorsicht: bezieht sich fast immer auf eine andere Systemversion
- Kurzversion mit Kurzbeispielen auf den **tldr-pages**: <https://tldr.sh>
- Viele Befehle bieten eine Hilfe-Option (meist sehr kanpp): **-h** oder **--help**
- Kurzbeschreibung eines Befehls: **whatis Befehlsname**
- Stichwortsuche in Manual-Pages: **apropos Stichwort**
- Allgemeines (etwas angestaubtes) Unix-Informationssystem:
info Befehlsname
- Aber wie findet man überhaupt den passenden Befehl?
Tipp: nach „Linux Befehlsreferenz“ googeln, z.B.: <http://www.helmbold.de/linux/>
ist recht brauchbar

Agenda



Linux 1: Einführung

- Was ist Linux?
- Unix-Dateisystem
- Erste Schritte mit der Shell
- Selbsthilfe



Editor vi

- **vi** ist einer der traditionellen Unix-Editoren und als modaler Editor ungewohnt
- Mode bestimmt, wie Tastatureingaben interpretiert werden:
 - **Normal-Modus** (Default): Tastatureingaben sind Befehle
 - **Einfüge-Modus**: Text eingeben wie von anderen Editoren gewohnt
 - **Kommandozeilen-Modus**: Dateioperation wie Speichern, Beenden
- Moduswechsel:
 - **i** (insert before cursor) oder **a** (...after...) -> Einfüge-Modus
 - **ESC** -> zurück in Normal-Modus
 - **:** (im Normal-Modus) -> Kommandozeilen-Modus
- **Normalmodus**: Cursor bewegen **hjk_l**, Zeile löschen mit **dd**, Undo **u**, Suche **/**
- **Kommandozeilen-Modus**: speichern (write) **w**, beenden (quit) **q**, beenden ohne Speichen **q!**
- Wichtigste Eingabesequenz: **ESC:q!**

Dateitypen

Dateityp	Beschreibung	Erzeugung
Directories	Verzeichnisse, Ordner	mkdir
Ordinary Files	Einfache Dateien	touch
Special Files	Spezielle Dateien, Gerätedateien	mknod
Named Pipes	Warteschlange, FIFO	mkfifo
Symbolic Links	Verweise, Verknüpfungen	ln -s

Dateityp	ls -l	ls -p
Directories	d	/
Ordinary Files	-	
Special Files	b, c	
Named Pipes	p	
Symbolic Links	l	@
Sockets	s	=

Zeitstempel von Dateien

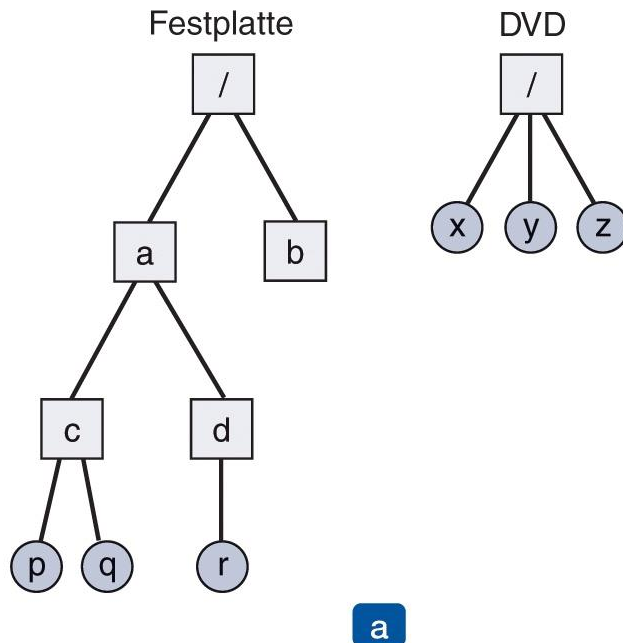
Typ	Beschreibung	Anzeige mit	Ändern mit
Modification Time	Letzte Änderung	ls -l	-mt
Access Time	Letzter Zugriff	ls -lu	touch -at
Change Time	Letzte Änderung der I-Node	ls -lc	

Mounten von Dateisystemen

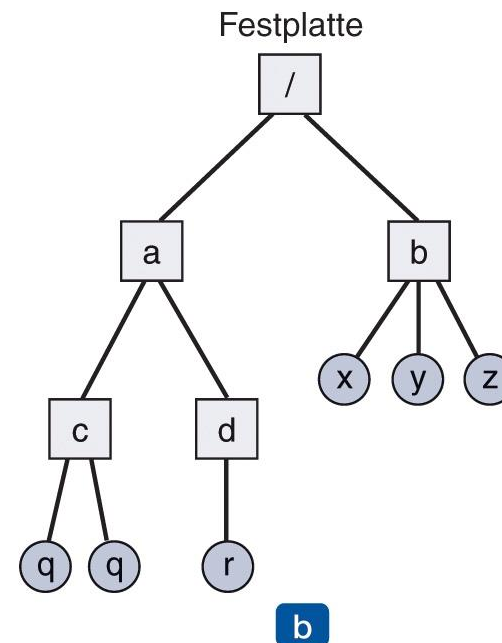
- Das Unix Dateisystem ist hierarchisch und wie *ein logisches Laufwerk* aufgebaut.
- Ein *physikalisches Laufwerk* kann aus *mehreren* logischen Laufwerksbereichen bestehen.
- Mehrere logische Laufwerke können durch den Systemaufruf *mount* zu einer gemeinsamen Hierarchie (Baum) zusammengesetzt werden.
- Befehle zur Verzeichnishierarchie:
 - Zusammensetzen mit *mount*
 - Freigeben mit *umount*
 - Zusammensetzen beim Systemstart durch Einträge in */etc/fstab*
 - Plattenverbrauch: *du*
 - Plattenplatz aufgeteilt nach Partitionen: *df*

Mounten von Dateisystemen

(a) Getrenntes Dateisystem
(vor dem mounten)



(b) nach dem Einhängen



`mount /dev/dvd /b`